



INSTITUTO POLITÉCNICO NACIONAL

Escuela Superior de Cómputo

ESCOM



Materia: Bases de Datos

Profesor: Gabriel Hurtado Aviles

Grupo: 3CV4

Título:

Práctica 1: "Modelo Entidad Relación"

Miembros de Equipo:

Gonzalez Carapia Brandon Uziel

Introducción

A lo largo de esta práctica se resolverán algunas preguntas así como poner en práctica lo visto hasta el momento en clases y poniendo en práctica lo que se llegue a investigar, con el objetivo de familiarizarse con un sistema como es el Git/Github realizando algunos ejercicios y aplicándolo en ciertos casos de nuestro interés.

Práctica 1: "Modelo Entidad Relación"

Objetivo: Que usted describa los sistemas de base de datos a partir de sus aspectos básicos, construya el entorno de trabajo que empleará durante todo el semestre —control de versiones y un sistema gestor de bases de datos ejecutado en un contenedor—, sustente su investigación con literatura científica arbitrada y aplique lo anterior al modelado conceptual de un problema real. Toda la evidencia se entrega en un repositorio de Git.

Esta práctica cubre la Unidad Temática I completa y, al mismo tiempo, establece la forma de trabajo del resto del curso. Se divide en cinco ejercicios: los dos primeros construyen el entorno técnico, el tercero y el cuarto sustentan la parte conceptual, y el quinto la aplica a un caso concreto. La entrega no es un archivo suelto: es un repositorio de Git donde quedará registrado cómo trabajó usted, no solo el resultado final.

Ejercicio 1. Control de versiones con Git y GitHub

Parte A. Investigación. Redacte al menos una cuartilla que responda lo siguiente:

1. Qué es un sistema de control de versiones y qué problema concreto resuelve en un trabajo en equipo.

Un sistema de control de versiones (VCS) es una herramienta de software diseñada para registrar los cambios realizados sobre un conjunto de archivos en un periodo de tiempo. Su propósito principal es permitir la recuperación de versiones específicas de un programa o proyecto, auditar las modificaciones efectuadas y facilitar la colaboración coordinada entre múltiples desarrolladores y equipos de trabajo.

2.Cuál es la diferencia entre Git y GitHub. No son sinónimos.

Git es un sistema de control de versiones distribuido, que ayuda en el mantenimiento de una gran cantidad de código, y que ofrece ciertos servicios a equipos compuestos por diversos programadores y GitHub funciona como un servidor en el que es posible almacenar repositorios de proyectos, agregando varias características para efectuar una gestión tanto de código como de un proyecto.

3. Defina, con sus palabras y con un ejemplo: repositorio, confirmación (commit), rama (branch), fusión (merge), conflicto de fusión, Pull Request, archivo .gitignore y archivo README.

- Repositorio: Es como una bóveda de datos que almacena los archivos de un proyecto junto con un historial de operaciones o cambios aplicados por algún miembro del equipo que esté trabajando. *Ejemplo: El directorio practical-bd/ que contiene todo el código y la carpeta oculta .git .*
- Confirmación(commit): Es como un cambio hecho por algún miembro en un determinado tiempo en el que guarda un “hasheo” con la información del cambio hecho. *Ejemplo: git commit -m "code:Cambio en notas de sintetizador" .*
- Rama(Branch):Es una rama de un árbol por así decirlo que apunta a un commit o cambio específico y que permite trabajar sobre esa sin alterar la rama principal llamada main o master. *Ejemplo: La rama Carapia creada para configurar el entorno del mapa y de los npc's.*
- Fusión(Merge):Función que combina un cambio de un commit a otro cambio,por ejemplo en mi proyecto cuando teníamos que poner o agregar funcionalidades ponemos el merge del cambio v3 al cambio main. *Ejemplo: Incorporar los cambios terminados de Carapia dentro de main .*
- Conflicto de fusión:Según yo era cuando nos daba el github el error de compatibilidad porque los cambios no se podían unir nos paso mucho cuando modificamos la misma función y nos dijeron que era porque pues el git no sabia cual agarrar como línea final por así decirlo.
- Pull Request:Es como la confirmación para integrar los cambios de una rama a otra. *Ejemplo: Un PR solicitando revisar el modelo ER antes de unirlo a la rama main .*
- Archivo .gitignore: Es el archivo de texto que especifica los patrones de búsqueda para indicar a git que archivos o directorios no deben de ser rastreados por la plataforma. *Ejemplo: El archivo *.log. del programa*
- Archivo README: Era como un manual de ejecución de un proyecto en el que integra lo necesario para usarlo,así como lo necesario para su instalación en un medio como VS. *Ejemplo: El archivo README.md que contiene la ficha técnica de la Práctica 1.*

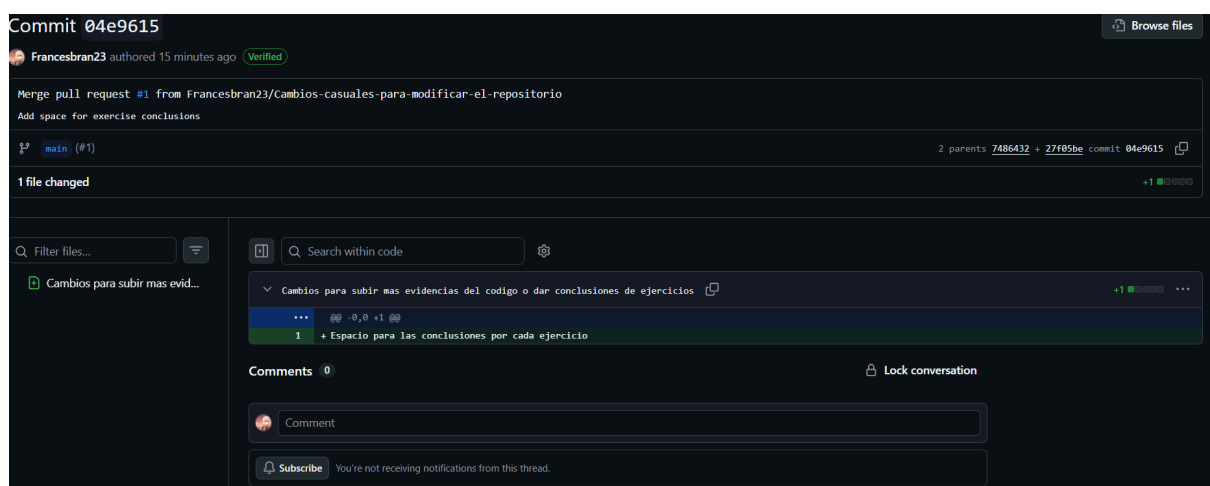
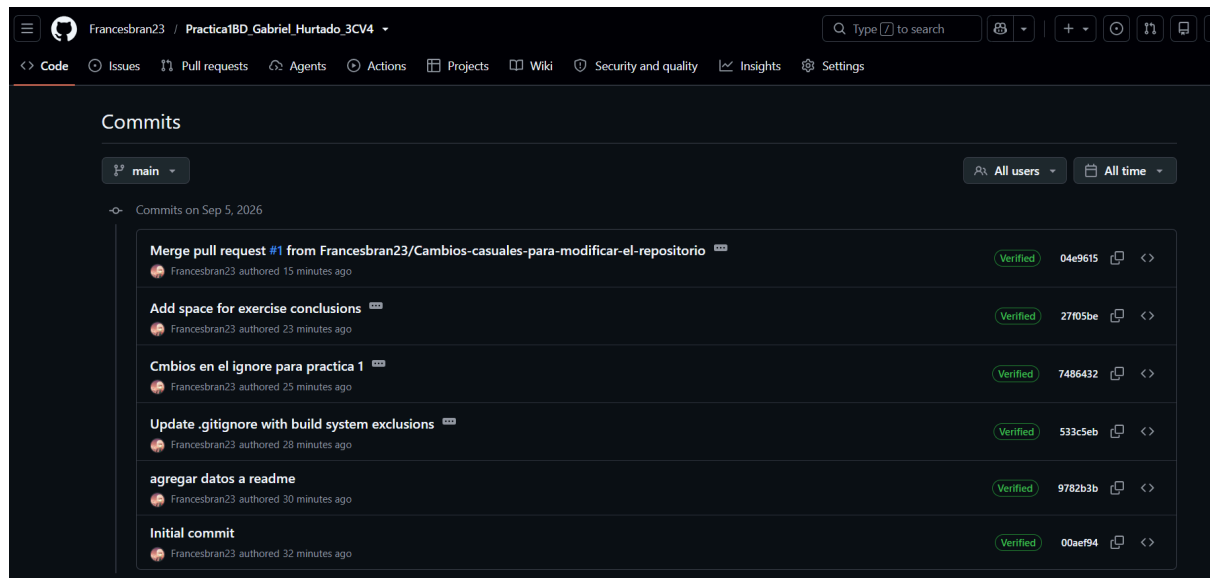
4. En qué consiste un flujo de trabajo basado en ramas y por qué el código se revisa entre pares antes de fusionarse.

Un Git Flow consiste en establecer una regla operacional de que la rama principal llamada main o master siempre debe de contener código estable y listo para producir y todos los cambios o experimentaciones deben de ser desarrollados en ramas secundarias.

La revisión entre códigos pares es una práctica esencial donde otros miembros del equipo examinan minuciosamente el código propuesto en el Pull Request antes de su fusión, por lo que se asegura la calidad técnica del producto previniendo así vulnerabilidades en el sistema.

Parte B. Práctica. Realice lo siguiente:

1. Cree una cuenta en GitHub, si aún no la tiene, y un repositorio para esta práctica.
 2. Agregue un archivo README.md con su nombre completo, grupo, carrera y un índice de la práctica.
 3. Agregue un archivo .gitignore adecuado al contenido del repositorio.
 4. Realice al menos cinco confirmaciones con mensajes descriptivos. Un mensaje como «cambios» no cuenta.
 5. Cree una rama, haga en ella un cambio, abra un Pull Request, descríballo y fusiónelo.
- Evidencia: la dirección del repositorio, una captura de la salida de git log --online --graph --all y una captura del Pull Request ya fusionado.



 Francesbran23 Merge pull request #1 from Francesbran23/Cambios-casuales-para-modifi... 04e9615 · 2 minutes ago 6 Commits		
 .gitignore	Cmbios en el ignore para practica 1	12 minutes ago
 Cambios para subir mas evidencias del codigo ...	Add space for exercise conclusions	10 minutes ago
 README.md	agregar datos a readme	17 minutes ago

 **Open** Francesbran23 wants to merge 1 commit into `main` from `Cambios-casuales-para-modificar-el-repositorio`

Conversation 0
Commits 1
Checks 0
Files changed 1
+1


Francesbran23 commented 6 minutes ago
 Owner

Espacio para las conclusiones por cada ejercicio

  Add space for exercise conclusions Verified 27f05be



Checking for the ability to merge automatically...

Hang in there while we check the branch's status.

Merge pull request
 You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)


Add a comment

Write Preview

H B I
 












































Add your comment here...

Markdown is supported
Paste, drop, or click to add files

Close pull request
Comment

Reviews
 No reviews
 Still in progress? [Convert to draft](#)

Assignees
 No one—assign yourself

Labels
 None yet

Projects
 None yet

Milestone
 No milestone

Development
 Successfully merging this pull request may close these issues.
 None yet

Notifications
 Customize

Unsubscribe

 You're receiving notifications because you authored the thread.

1 participant

Conclusión

Gonzalez Carapia Brandon Uziel:

En el desarrollo de esta primera práctica se logró construir y consolidar el entorno técnico de trabajo indispensable para la materia de Bases de Datos, integrando herramientas fundamentales como el control de versiones y la virtualización ligera por contenedores y mediante el uso de Git y GitHub, se trabajó con flujos basados en ramas (*Git Flow*), lo cual garantiza la estabilidad de la rama principal (*main*) y facilita la colaboración en equipo mediante revisiones de código (*Pull Requests*) y el registro detallado de cambios mediante *commits* descriptivos.

Bibliografía

- Chacon, S., & Straub, B. (2014). *Pro Git* (2.^a ed.). Apress.
<https://git-scm.com/book/en/v2>
- GitHub. (2024). *About pull requests*. GitHub Docs.
<https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests>
- GitHub. (2024). *Getting started with GitHub*. GitHub Docs.
<https://docs.github.com/en/get-started>
- Loeliger, J., & McCullough, M. (2012). *Version Control with Git: Powerful tools and techniques for collaborative software development* (2.^a ed.). O'Reilly Media.
- Sommerville, I. (2016). *Software Engineering* (10.^a ed.). Pearson.

